CSE113 Homework 1 Report

Rachel (Younwoo) Park

Dept. of Systems Management Engineering
Sungkyunkwan University

October 15, 2024

Abstract

This is the report for the first homework assignment of CSE113: Parallel and Concurrent Programming.

Contents

1 Part 1 Performance Analysis **2**
1.1 Analysis of Sequential Loop Unrolling 2
1.2 Analysis of Interleaved Loop Unrolling 2
1.3 Difference between Local and Server Results 2

2 Part 2: Unrolling Reduction Loops for ILP **3**
2.1 Analysis of Reduction Loop Unrolling 3
2.2 Difference between Local and Server Results 3

3 Part 3: SPMD Parallel Programing Using C++ Threads **4**
3.1 Strategy for Computing Array C 4
3.2 Results Analysis 4

1 Part 1 Performance Analysis

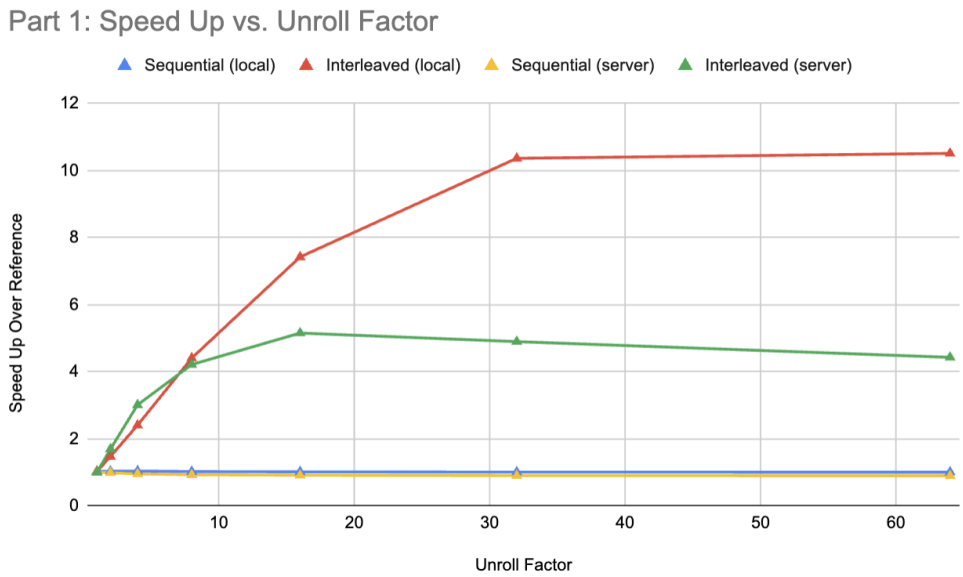

Figure 1: Part 1: Speed Up vs. Loop Unrolling Factor

1.1 Analysis of Sequential Loop Unrolling

In figure 1, sequental loop unrolling did not show much improvement over the reference loop. Even at higher unroll factors, the speed up remained close to 1. One may expect the performance to improve because the loop unrolling reduces the number of iterations in the outermost loop. However, it is important to note that the instructions in each sequence are dependent on each other. Therefore, the processor is still forced to execute each instruction in order, limiting the potential for parallel execution.

1.2 Analysis of Interleaved Loop Unrolling

In figure 1, the interleaved loop unrolling shows significant performance improvement compared to the reference loop. The speed up of the interleaved loop unrolling improves noticeably as the unrolling factor increases, to the point where the speed up was over 10 times faster locally compared to the reference loop (at unrolling factor of 64). This observation is consistent with the concept of Instruction Level Parallelism (ILP) – by spacing out dependent instructions the CPU can take advantage of its pipelines to execute multiple instructions simultaneously.

While the sequential loop unrolling placed the dependent instructions consecutively, the interleaved loop unrolling placed independent instructions in between dependent instructions. Hence, it displayed a much better performance improvement compared to the reference loop and the sequential loop unrolling.

1.3 Difference between Local and Server Results

While most of the local and server results exhibited similar trends, there was one key difference in the interleaved loop unrolling's performance. The speed up for the server reaches its maximum at an unroll factor of 16, while it continues to increase for the local machine (maximum reached at unroll factor of 64). A possible explanation for this discrepancy is the difference in hardware. The server may have less cores, different cache sizes, or less efficient pipelining compared to the local machine. It is likely that the server had less cores because the speed up for the server was lower than the local machine for most unroll factors. Furthermore, depending on the cache size, the server may have had to access the main memory more frequently, which would have slowed down the performance. Other reasons could include that the server encountered register spilling while the local machine did not.

2 Part 2: Unrolling Reduction Loops for ILP

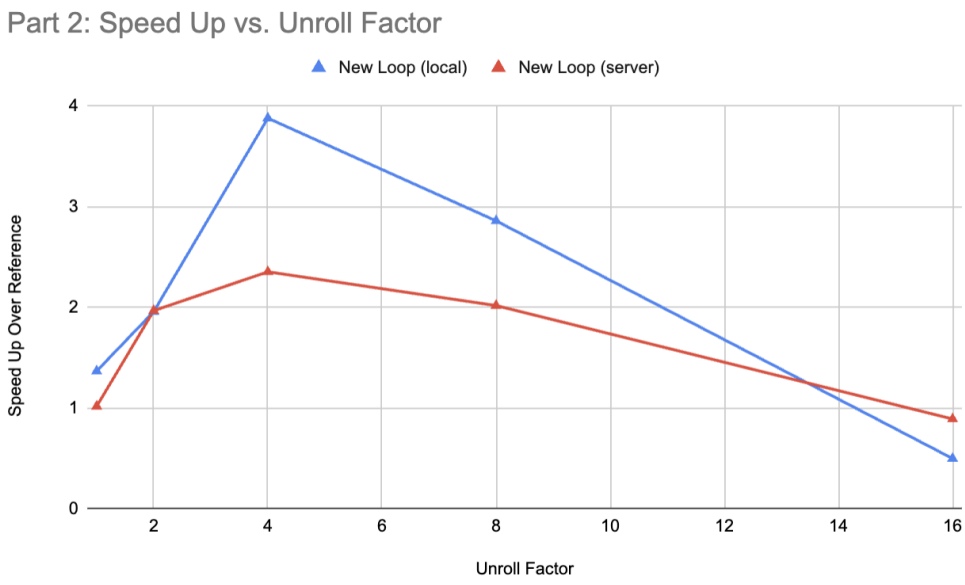

Figure 2: Part 2: Speed Up vs. Loop Unrolling Factor

2.1 Analysis of Reduction Loop Unrolling

In figure 2, a notable trend is that the speed up initially increases as the unroll factor increases, hitting its peak at an unroll factor of 4. After this point, the performance begins to decline in both the local machine and the server until it actually becomes slower than the reference loop.

The initial increase was expected because unrolling reduces loop overhead and enables the processor to handle multiple independent computations simultaneously, taking full advantage of instruction-level parallelism (ILP). However, as the unroll factor increases beyond 4, other factors, such as cache size and memory bandwidth, seem to limit further improvements. The unroll factors were also the number of accumulators (local variables) within the function. As the number of accumulators increased, the number of registers used also increased. This could have caused register spilling, which would have slowed down the performance.

2.2 Difference between Local and Server Results

The difference between the local ans server results once again highlight how hardware plays a critical role in determining the effectiveness of optimizations. As register spilling occurs, variables had to be stored in the main memory. From the local machine's much steeper slopes, one can infer that the local machine took more clock cycles to access the main memory but less clock cycles to access the cache compared to the server.

3 Part 3: SPMD Parallel Programing Using C++ Threads

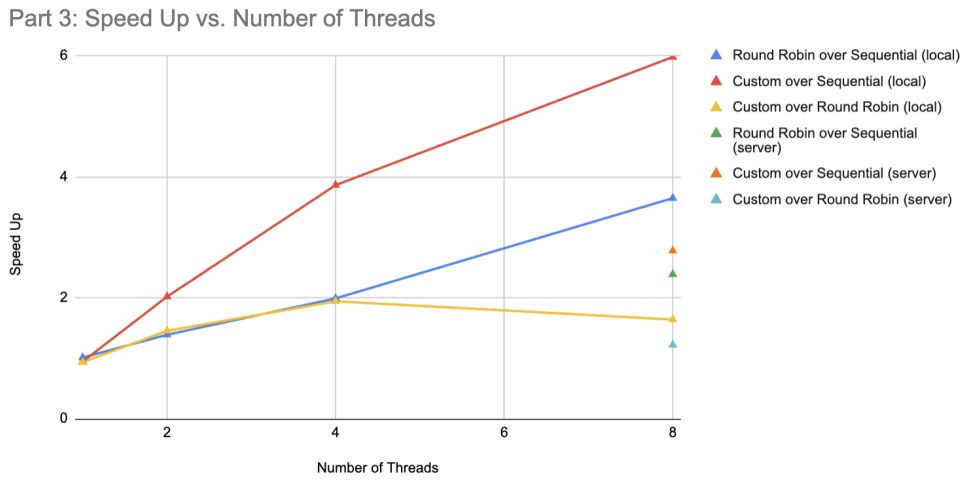

Figure 3: Part 3: Speed Up vs. Number of Threads

3.1 Strategy for Computing Array C

For the `custom_increment` function, I assigned each thread to a contiguous block of the array to operate on in order to take advantage of cache locality. If two different threads work on the same cache line, the cache line would have to be invalidated and reloaded, creating significant overhead. By divindg the array by the number of threads and assigning each thread to `(array_size / num_threads)` elements, the threads could work on their own cache lines without interfering with each other. Any leftover elements were assigned to the last thread to ensure that all elements were accounted for.

3.2 Results Analysis

Both on the local machine and the server, the custom increment function showed greate performance with 6x speed up in the local machine when operating with 8 threads. This result aligns with expectations as threads working on contiguous blocks of the array would provide bettwe cache locality. In contrast, since round robin leads to threads working on elements in different locations on the array, it has poorer cache locality and higher chances of cache misses.

Once again, the local machine performed better than the server. This is likely due to the hardware differences between the two machines. The local machine may have had more cores, larger cache sizes, or more efficient pipelining compared to the server. THe difference between custom increment and round robin increment over the reference is less pronounced in the server. This may also be due to the server's software capabilities, in which its thread scheduling policy or context switching may have been less efficient than the local machine.